Atelier de professionnalisation

Partie 5.1 : Résolution de bug

Application mobile de gestion des notes de frais — GSB

Introduction

L'équipe de test a observé que l'authentification ne redirige pas vers le Dashboard : après une connexion réussie, l'utilisateur reste sur l'écran Home. C'est un bug qui empêche de mettre votre application en production. Dans cet atelier, vous allez l'observer, comprendre sa cause, puis le corriger avec Stack.Group et navigationKey.

> **Travail à faire**
> En plus de modifier votre code, le commiter et le pousser sur Github, vous réaliserez un compte-rendu avec les réponses aux questions des encadrés « A vous de réfléchir ».

Étape 1 : Observer le bug

Lancez l'application puis connectez-vous avec un compte valide. Observez l'écran affiché une fois la connexion réussie, puis répondez aux questions.

> **À vous de réfléchir**
> 1. Dans AppNavigator, quels écrans sont déclarés quand user vaut null ? Et quand user contient un utilisateur connecté ?
> 2. À la connexion, l'écran Login n'est plus déclaré : React Navigation le retire de la pile. Quel écran se retrouve alors affiché, et pourquoi ?
> 3. Quel message d'erreur obtient-on quand on appelle navigation.navigate("Dashboard") dans le code de connexion (dans la fonction handleSubmit() de LoginScreen) ?

Étape 2 : Comprendre Stack.Group

Stack.Group est un composant de React Navigation qui regroupe plusieurs écrans d'un même Stack.Navigator. Il n'affiche rien lui-même : c'est un regroupement logique, qui sert à trois choses.

- Appliquer des options communes à tous les écrans du groupe grâce à screenOptions (style de l'en-tête, présentation en fenêtre modale...).
- Déclarer plusieurs écrans d'un seul bloc, par exemple dans une condition user ? (...) : (...), où l'on ne peut renvoyer qu'un seul élément. Contrairement au fragment <>, le groupe accepte des propriétés.
- Réinitialiser la pile grâce à navigationKey.

```
<Stack.Group screenOptions={{ headerShown: false }}>
  <Stack.Screen name="A" component={A} />
  <Stack.Screen name="B" component={B} />
</Stack.Group>
```

navigationKey est la clé associée à un groupe. Quand cette clé change, les écrans de l'ancien groupe sont retirés de la pile, et ceux du nouveau groupe sont montés : on repart du premier écran déclaré dans le nouveau groupe.

> **Pourquoi on reste sur l'écran d'accueil Home**
> Dans notre application, l'écran Home existe dans l'état connecté comme dans l'état déconnecté. Sans clé, React Navigation reconnaît le même écran Home d'un état à l'autre et le conserve dans la pile, ce qui produit le bug observé. Avec deux clés différentes, il considère qu'il s'agit de deux écrans distincts et repart de zéro.

Étape 3 : Corriger AppNavigator

Modifiez AppNavigator dans App.js pour utiliser deux groupes, un par état de connexion :

```
<Stack.Navigator>
  {/* navigationKey : quand on change de groupe (connexion / déconnexion),
      la pile est vidée et on démarre sur le 1er écran du nouveau groupe */}
  {user ? (
    <Stack.Group navigationKey="connecte">
      <Stack.Screen name="Dashboard" component={DashboardScreen} />
      <Stack.Screen name="FraisAdd" component={FraisAddScreen} />
      <Stack.Screen name="FraisEdit" component={FraisEditScreen} />
      <Stack.Screen name="Home" component={HomeScreen} />
    </Stack.Group>
  ) : (
    <Stack.Group navigationKey="invite">
      <Stack.Screen name="Home" component={HomeScreen} />
      <Stack.Screen name="Login" component={LoginScreen} />
    </Stack.Group>
  )}
</Stack.Navigator>
```

Modifiez votre code de LoginScreen pour qu'il corresponde au code suivant, et assurez vous notamment que la fonction handleSubmit() ne fait plus appel à navigation.navigate() :

```
import { useState } from "react";
import { useAuth } from "../context/AuthContext";
import { View, Text, TextInput, Pressable, Alert } from "react-native";
import Navbar from "../components/Navbar";
import { loginStyles as styles } from "../styles/loginStyles.js";

export default function LoginScreen() {
  // 1. États locaux pour les champs du formulaire
  const [login, setLogin] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [submitting, setSubmitting] = useState(false);

  //2. Récupération de la fonction loginUser depuis le contexte d'authentification
  const { loginUser } = useAuth();

  // 3. Déclaration de la fonction handleSubmit pour gérer la soumission du formulaire
  const handleSubmit = async () => {
    setSubmitting(true); // Indique que la soumission est en cours
    setError(""); // Réinitialise l'erreur avant de tenter la connexion
    try {
      const data = await loginUser(login, password); // Appel de la fonction loginUser du
contexte d'authentification
    } catch (e) {
      setError(e.message); // Met à jour l'état de l'erreur pour l'afficher à l'utilisateur
    } finally {
      setSubmitting(false); // Réinitialise l'état de soumission après la tentative de
connexion
    }
  };

// 4. Render du composant LoginScreen avec le formulaire de connexion
  return (
    <View style={{ flex: 1 }}>
      <Navbar />
      <View style={styles.container}>
        <Text style={styles.title}>Connexion</Text>
```

```
        <View>
          <Text>Login :</Text>
          <TextInput
            style={styles.input}
            value={login}
            autoCapitalize="none"
            onChangeText={setLogin}
          />
        </View>

        <View>
          <Text>Mot de passe :</Text>
          <TextInput
            style={styles.input}
            value={password}
            onChangeText={setPassword}
            secureTextEntry
          />
        </View>

        <Pressable style={styles.button} onPress={handleSubmit} disabled={submitting}>
          <Text style={styles.buttonText}>{submitting ? "Connexion..." : "Se
connecter"}</Text>
        </Pressable>
        {error ? <Text style={{ "color": "red" }}>{error}</Text> : null}
      </View>
    </View>
  );
}
```

Testez ensuite les scénarios suivants :

- Après la connexion, l'écran Dashboard s'affiche directement.
- Sur Dashboard, le bouton retour du téléphone ne ramène pas à l'écran de connexion.
- Après la déconnexion, l'écran Home s'affiche.
- L'ajout, la modification et la suppression d'une note de frais fonctionnent toujours.

> **À vous de réfléchir**
>
> 1. Pourquoi Home est-il déclaré dans les deux groupes, plutôt qu'une seule fois avant la condition ?
> 2. Expérience : supprimez les deux navigationKey, relancez l'application et connectez-vous. Que constatez-vous ? Remettez-les, puis expliquez avec vos mots le rôle de la clé.
> 3. Pourquoi Dashboard est-il déclaré en premier dans le groupe « connecte » ?
> 4. Un utilisateur non connecté peut-il atteindre FraisAdd ? Cette organisation des écrans suffit-elle à protéger les données des notes de frais ? Justifiez.